

Document Q&A RAG Bot

Complete Architecture Guide: From Scratch to Advanced Production

This enterprise engineering guide covers the entire technical architecture of the Retrieval-Augmented Generation (RAG) system built with Pinecone Serverless Vector Database, Google Gemini 3.5 Flash, and Streamlit.

It details everything from basic foundational concepts (chunking, embeddings, vector search) to advanced production engineering (real-time SSE streaming, multi-turn context rephrasing, multimodal vision OCR, and automatic rate-limit failovers).

- **Vector Database:** Pinecone Cloud Serverless (AWS us-east-1 | 768-dim)
- **Embedding Model:** Google gemini-embedding-001 (Task-Type Specialized)
- **Generative LLM:** Google Gemini 3.5 Flash (SSE Real-Time Streaming)
- **Vision OCR:** Gemini Flash Multimodal Vision (Scanned/Handwritten PDFs)
- **User Interface:** Interactive Web App with Conversational Memory (Streamlit)
- **Repository:** github.com/ANAND9051/rag-bot

Author & Engineer: Anand | System: rag_bot

Generated for: Full Stack AI Engineering Reference

Complete Technical Architecture Guide: Document Q&A RAG Bot

From Fundamentals to Production-Grade Vector Architecture

Table of Contents

1. Level 1: Fundamentals - Why RAG Exists
 2. Level 2: The Document Ingestion & Chunking Pipeline
 3. Level 3: Vector Embeddings & High-Dimensional Geometry
 4. Level 4: Pinecone Serverless Vector Database
 5. Level 5: Query Execution & Grounded Generation
 6. Level 6: Production & Enterprise Upgrades in this Codebase
 7. System Architecture Diagram
 8. Codebase Directory & Component Map
-

Level 1: Fundamentals - Why RAG Exists

The Core Problem with Standard LLMs

Large Language Models (LLMs) like GPT, Gemini, or Claude are trained on massive public web datasets. However, they suffer from three fundamental limitations in business environments:

1. Knowledge Cutoff: They know nothing about events or data after their training date.
2. Private Data Blindness: They cannot read your internal files (PDFs, course slides, proprietary manuals, private codebase).
3. Hallucination: When an LLM doesn't know an answer, it frequently generates convincing but fabricated statements.
4. Context Window Costs & Latency: Stuffing an entire 500-page book or company wiki into the prompt every time is slow, expensive, and frequently exceeds token limits.

The Solution: RAG (Retrieval-Augmented Generation)

RAG solves this by acting like an "Open Book Exam" for the AI:

[Traditional LLM] > Answers from internal memory > Risk of Hallucination

[RAG Architecture] > Searches your documents first > Answers ONLY from retrieved facts

RAG consists of three distinct phases:

1. Retrieval: When the user asks a question, search your documents to pull out the 3 to 5 most relevant paragraphs.
 2. Augmentation: Combine those retrieved paragraphs with the user's question into a strict prompt.
 3. Generation: The LLM reads the retrieved context and drafts an accurate, cited answer.
-

Level 2: The Document Ingestion & Chunking Pipeline

Before you can search documents, they must be converted into searchable units. This happens in ingest.py.

1. Multi-Format Text Extraction

Real-world enterprise documents come in different states:

- o Digital PDFs / Text / Markdown: Extracted directly using pypdf or UTF-8 text streams.
- o Scanned & Image-Based PDFs: If a PDF has zero selectable text (e.g., scanned handwritten notes like C Language Notes.pdf), the pipeline automatically activates Gemini Multimodal Vision OCR to read diagrams, code, and handwriting, then caches the transcription in a local .extracted.md file so future runs take 0.1 seconds.

2. Recursive Boundary-Aware Chunking

Why can't we embed an entire 50-page document as a single vector?

- o An embedding vector has a fixed dimension (768 numbers). If you squeeze 50 pages into 768 numbers, subtle details are lost.
- o Specific answers are usually found in 1-2 paragraphs.

In chunking.py, the RecursiveTextSplitter implements smart boundaries:

- o Hierarchy of Separators: ["\n\n", "\n", ". ", " ", "", ""]
- o First, it tries to split on paragraph boundaries (\n\n).
- o If a paragraph is still larger than chunk_size (700 characters), it splits on sentences (.).
- o If a sentence is still too long, it splits on words ().
- o It never cuts a word in half.
- o Chunk Overlap (150 characters):
 - o Sentences at the boundary of a cut are repeated at the beginning of the next chunk so context is never severed mid-thought.

Level 3: Vector Embeddings & High-Dimensional Geometry

Computers cannot compare text strings for semantic meaning using keywords alone (e.g., "automobile" and "car" have zero letters in common, but identical meaning). Vector embeddings solve this mathematically.

1. What is an Embedding?

An embedding model converts a piece of text into an array of 768 floating-point numbers:

$$\vec{v} = [-0.0421, 0.0892, 0.0135, \dots, -0.0611] \in \mathbb{R}^{768}$$

Each number represents a latent semantic concept (e.g., syntax, formality, domain, topic). Texts with similar meanings end up close to each other in 768-dimensional space.

2. Task-Type Specialization

In gemini_client.py, we use Google's gemini-embedding-001 with asymmetric task types:

- o When ingesting documents: task_type="RETRIEVAL_DOCUMENT"
- o When searching a user question: task_type="RETRIEVAL_QUERY"

[!NOTE]

Questions and answers look linguistically different. Specifying RETRIEVAL_QUERY vs RETRIEVAL_DOCUMENT trains the embedding space so that a question vector aligns directly with the vector of the document paragraph that answers it.

3. Cosine Similarity

To measure how closely a query matches a stored document chunk, we compute the cosine of the angle between their

vectors:

$$\text{Cosine Similarity}(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| |\vec{B}|}$$

- o 1.0: Identical semantic meaning
- o 0.7 - 0.9: High relevance
- o < 0.5: Irrelevant or unrelated

Level 4: Pinecone Serverless Vector Database

While traditional databases (SQL, Postgres) excel at exact matching (WHERE id = 5), finding the closest 768-dimensional vectors across thousands of chunks requires specialized index structures known as Approximate Nearest Neighbor (ANN) search.

In vector_db.py:

- o Hosted Cloud Serverless: Runs on Pinecone's managed infrastructure on AWS us-east-1.
- o Index Dimensions: 768 dimensions with Cosine distance metric.
- o Payload Metadata Storage:

```
json
{
  "id": "java_course.pdf_chunk_3_a9b1c2",
  "values": [-0.021, 0.045, ...],
  "metadata": {
    "text": "Core Java (J2SE) is used for standalone desktop software...",
    "source": "docs/java_course.pdf",
    "page": 2,
    "chunk_index": 3
  }
}
```

Because chunk text and document metadata are stored directly in Pinecone's metadata payload, retrieval is a single network hop with zero need for a separate SQL lookup.

Level 5: Query Execution & Grounded Generation

When a user asks: "What are the flavours of Java?", the following sequence occurs in query.py and app.py:

User Query > Embed (Query Vector) > Pinecone Top-K Search > Context Assembly > Gemini Grounded Generation

1. Grounded Context Construction

The retrieved chunks are formatted into numbered source blocks:

text

=== CONTEXT ===

[Source #1 | File: java_course.pdf | Page 1 | Cosine Score: 0.6943]

Flavours of Java: Core Java (J2SE), Advance Java (J2EE), Android Java (J2ME).

2. The Strict Anti-Hallucination Guardrail

The LLM is prompted with strict instructions:

text

You are a precise, reliable enterprise Q&A assistant.

Answer the user's question using ONLY the factual context provided below.

If the context does not contain enough information, explicitly state:

"I do not have enough information in the provided documentation to answer that question."

Do not extrapolate beyond what is documented.

If a user asks "What is the weather in Tokyo?", the model detects no supporting context and safely refuses to hallucinate.

Level 6: Production & Enterprise Upgrades in this Codebase

Your project includes features that elevate it beyond prototype code into an enterprise-ready system:

1. Real-Time Token Streaming (SSE)

Instead of waiting 3-5 seconds for the entire answer to generate, `generate_answer_stream` uses Server-Sent Events (`alt=sse`) over HTTP stream connections. Words render on the user's screen in real time as they are generated.

2. Conversational Memory & Contextual Query Reformulation

If a user asks:

- o Turn 1: "What is Java SE?"
- o Turn 2: "Which devices does it run on?"

A standard vector database would search for the phrase "Which devices does it run on?" and fail, because "Java SE" isn't mentioned in Turn 2.

In `app.py`, `condense_query_with_history()` uses Gemini to rewrite Turn 2 into a standalone query:

```
$$\text{"What devices does Java SE run on?"}$$
```

This ensures accurate Pinecone retrieval while preserving multi-turn chat flow.

3. High-Capacity Model & Automatic Quota Fallback

Google's Free Tier assigns tight daily limits (e.g., 20 requests/day) to certain preview models.

- o The pipeline defaults to `gemini-3.5-flash`, which has high quota headroom.
- o If any model returns HTTP 429 (Resource Exhausted) or 503 (High Demand), the client automatically cascades through fallback models:

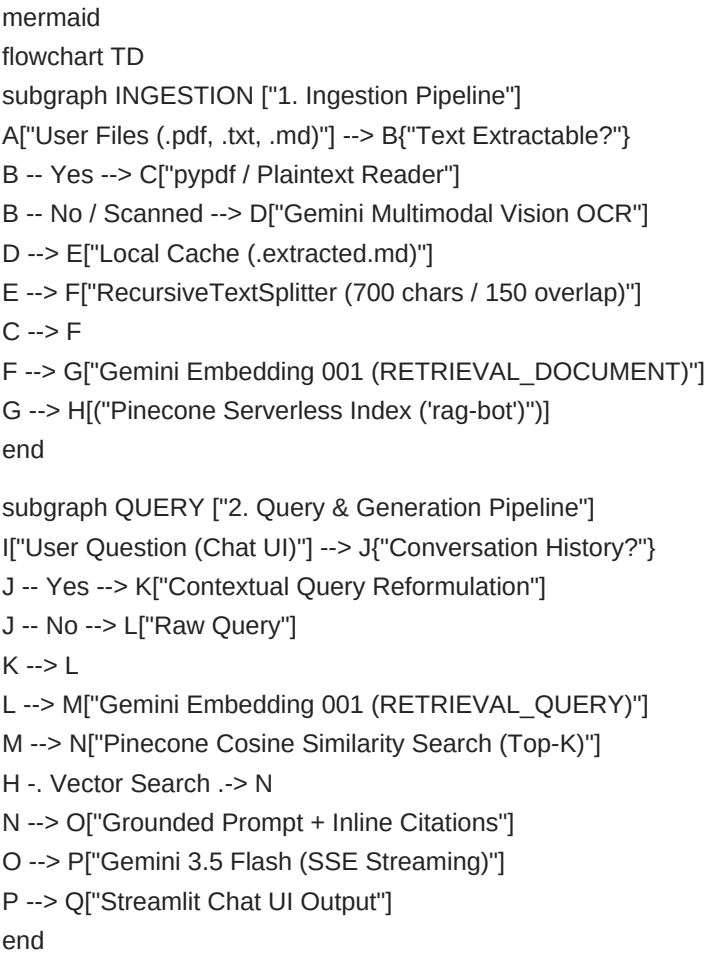
```
$$\text{gemini-3.5-flash} \rightarrow \text{gemini-3-flash-preview} \rightarrow \text{gemini-flash-latest}$$
```

- o Transient rate limits trigger automatic exponential backoff retries instead of crashing.

4. Interactive Streamlit Web UI (`app.py`)

- o Live Pinecone index health telemetry (live vector counts, AWS region, metric).
- o In-browser file uploader for `.pdf`, `.txt`, and `.md`.
- o Expandable citation accordions displaying similarity scores and exact text chunks.
- o Parameter sliders for Top-K retrieval, temperature, and model selection.

System Architecture Diagram



Codebase Directory & Component Map

File	Purpose	Key Classes / Functions
---	---	---
app.py	Full interactive web application (Streamlit)	ingest_files_ui(), condense_query_with_history(), format_context_for_llm()
vector_db.py	Pinecone cloud vector database client	PineconeVectorDB, .add_documents(), .search(), .clear()
gemini_client.py	Google Gemini API client (REST & SSE)	GeminiClient, .embed_text(), .generate_answer_stream(), .extract_pdf_multimodal()
chunking.py	Boundary-aware text splitter	RecursiveTextSplitter, .split_text()
ingest.py	CLI ingestion tool & multimodal OCR handler	extract_text_from_file(), ingest_documents()
query.py	Terminal-based interactive Q&A CLI	ask_question(), format_context()
requirements.txt	Environment dependencies	pinecone, streamlit, pypdf, python-dotenv, requests